No.	Python Basic Questions
	What is Python?
	Features of Python?
	Python Frameworks?
	Difference between Python 2 and Python 3
	What is an Interpreted language?
	What is a dynamically typed language?
	Is indentation required in python?
	What is PEP 8?
	File Extensions in Python?
	What is the difference between .py and .pyc files?
	Python Comments?
	What is Docstrings?
	how to get docstring in python
	Difference between Python comments and docstrings?

No.	Variables
	What is python Variables?
	what is Global Variables?
	what is global Keyword?
	How to get Id of any object?
	a=1, b=1 does both have same Id or not?
	What is None in Python
	Difference between None, NaN, and Null
	How is memory managed in Python?
	What is Scope in Python

No.	Operator Questions
	What is the operator?
	What is membership operators?
	What is Identity operators?
	Difference between '==' and 'is' Operator?
	What is ternary operator in Python?

🎯 Ques. What is Python?

- Python is a high-level, interpreted, general-purpose programming language.
- Python is an **interpreter** language. It means it executes the code line by line, which helps in debugging and testing.
- It was created by **Guido van Rossum**, and released in **1991**
- It is used for:
    - web development (server-side)
    - software development
    - mathematics
    - system scripting

🎯 Ques. Features of Python?

1. **Easy:-** Python is very easy to learn and understand; using this Python tutorial, any beginner can understand the basics of Python.
2. **Interpreted:-** It is interpreted(executed) line by line. This makes it easy to test and debug.
3. **Object-Oriented:-** The Python programming language supports classes and objects. We discussed these above.
4. **Free and Open Source:-** The language and its source code are available to the public for free; there is no need to buy a costly license.
5. **Portable:-** Since it is open-source, you can run Python on Windows, Mac, Linux or any other platform. Your programs will work without needing to the changed for every machine.
6. **GUI Programming:-** You can use it to develop a GUI (Graphical User Interface). One way to do this is through Tkinter.
7. **Large Library:-** Python provides you with a large standard library. You can use it to implement a variety of functions without needing to reinvent the wheel every time. Just pick the code you need and continue. This lets you focus on other important tasks.

🎯 Ques. Python Frameworks?

- **Web Development:** Django, Pyramid, Bottle, Tornado, Flask, web2py
- **GUI Development:** tkInter, PyGObject, PyQt, PySide, Kivy, wxPython
- **Scientific and Numeric:** SciPy, Pandas, IPython
- **Software Development:** Buildbot, Trac, Roundup
- **System Administration:** Ansible, Salt, OpenStack, xonsh

🎯 Difference between Python 2 and Python 3?

Feature	Python 2	Python 3
Release Year	2000	2008
Current Status	❌ End of Life (Jan 2020)	✅ Actively maintained
Print Statement	print "Hello"	print("Hello")
Default String	Type ASCII	Unicode (UTF-8)
Integer Division	5 / 2 = 2	5 / 2 = 2.5
Input Function	raw_input()	input()
Range Function	range() returns list	range() returns iterator
Error Handling	Old syntax	Improved syntax
Libraries Support	Limited (deprecated)	Full & modern support
Performance	Slower	Faster & optimized
Security Updates	❌ No	✅ Yes

- Print Function

```
# Python 2
print "Hello World"

# Python 3
print("Hello World")
```

- Division Behavior

```
# Python 2
print(5 / 2)   # Output: 2

# Python 3
print(5 / 2)   # Output: 2.5
```

- Input Handling

```
# Python 2
name = raw_input("Enter name: ")

# Python 3
name = input("Enter name: ")
```

- Unicode vs String (Very Important)

```
# Python 2
s = "hello"     # ASCII
u = u"hello"    # Unicode


# Python 3
s = "hello"     # Unicode by default,  Remove u""
```

- range() Behavior Changed

```
# Python 2
range(5)    # returns list

# Python 3
range(5)    # returns iterator
```

- xrange() Removed

```
# Python 2
xrange(10)

# Python 3
range(10)
```

- Exception Syntax Changed

```
# Python 2
except Exception, e:
    print e

# Python 3
except Exception as e:
    print(e)
```

- long Type Removed

```
# Python 2
x = 123L

# Python 3
x = 123 # Python 3 handles large integers automatically.
```

🎯 Ques. What is an Interpreted language?

- An Interpreted language **executes** its statements **line by line**. Languages such as Python, Javascript, R, PHP, and Ruby are prime examples of Interpreted languages.

🎯 Ques. What is a dynamically typed language?

- Type-checking can be done at two stages:-

1. **Static-** Data Types are checked before execution.
2. **Dynamic-** Data Types are checked during execution. Python is an interpreted language, executes each statement line by line and thus type-checking is done on the fly, during execution. Hence, Python is a Dynamically Typed Language.

🎯 Ques. Is indentation required in python?

- Indentation in Python means the **spaces** (or **tabs**) you put at the beginning of a line of code.
- Indentation is necessary for Python. It specifies a block of code. All code within loops, classes, functions, etc is specified within an indented block.
- It is usually done using four space characters. If your code is not indented necessarily, it will not execute accurately and will throw errors as well.

🎯 Ques. What is PEP 8?

- PEP stands for **Python Enhancement Proposal**.
- It is a set of rules that specify how to format Python code for maximum readability.
- PEP8 is a document that provides various guidelines to write the readable in python.
- PEP8 describe how the developers can write the beautiful code.

🎯 Ques. File Extensions in Python?

- **.py–** The normal extension for a Python source file
- **.pyc-** The compiled bytecode
- **.pyd-** A Windows DLL file
- **.pyo-** A file created with optimizations -> outdated ❌
- **.pyw-** A Python script for Windows
- **.pyz-** A Python script archive

.py

- .py is the Python source-code file written by the developer, while .pyc is the cached bytecode generated by Python from the .py file. .pyc is executed by the Python Virtual Machine (PVM).`

.pyc

- It contains Python bytecode, which is an intermediate representation executed by the Python Virtual Machine.
- .pyc files are commonly stored inside the **pycache** directory:

```
project/
│
├── main.py
└── __pycache__/
    └── main.cpython-312.pyc
```

Why .pyc exists?

- Mainly to **avoid recompiling the source to bytecode every time**, when the cached bytecode is still valid. This can make module loading faster.

🎯 Ques. What is the difference between .py and .pyc files?

`.py`	`.pyc`
.py files contain the source code of a program	**.pyc** file contains the bytecode of your program.
Human-readable	Mostly machine-readable/not intended for humans
We write our Python code here	Python generates it automatically
Example: `main.py`	Example: `main.cpython-312.pyc`
Executed after being compiled to bytecode	Bytecode is executed by Python Virtual Machine (PVM)

🎯 Ques. Python Comments?

- Comments are hints that we add to our code to make it easier to understand.
- Comments are short descriptions along with the code to increase its readability.
- **single Line Comments:-** Comments starts with a #, and Python will ignore them:

```
#This is a comment
print("Hello, World!")

print("Hello, World!") #This is a comment
```

Multi Line Comments(OR)Docstring

- To add a multiline comment you could insert a # for each line.

```
#This is a comment
#written in
#more than just one line
print("Hello, World!")
```

- you can add a multiline string (triple quotes) in your code, and place your comment inside it.

```
"""
This is a comment
written in
more than just one line
"""
print("Hello, World!")
```

🎯 Ques. What is Docstrings?

- In Python, docstrings are a way of documenting modules, classes, functions, and methods. They are written within triple quotes (""" or ''') and can span multiple lines.
- Python docstrings are strings used right after the definition of a function, method, class, or module. They are used to document our code.
- There are two main types of docstrings:
    - **Single-line docstrings:** Used for simple explanations that fit on one line.
    - **Multi-line docstrings:** Used for more detailed explanations, including parameter descriptions, return values, and exceptions.

Ques. how to get docstring in python?

- **Example and Accessing Docstrings:-**

```
class Calculator:
    """
    A simple calculator class to perform basic arithmetic operations.
    """
    def add(self, a, b):
        """
        add(a, b): Return the sum of two numbers.
        """
        return a + b

    def multiply(self, a, b):
        """
        multiply(a, b): Return the product of two numbers.
        """
        return a * b

# Accessing the class docstring
print(Calculator.__doc__) # Output:- A simple calculator class to perform basic
arithmetic operations.

# Accessing the method docstring
print(Calculator.add.__doc__)     # Output:- add(a, b): Return the sum of two
numbers.
print(Calculator.multiply.__doc__)  # Output:-  multiply(a, b): Return the product
of two numbers.
```

```
def square(n):
    '''Take a number n and return the square of n.'''
    return n**2

print(square.__doc__)   # Output:- Take a number n and return the square of n.
```

- Using the **doc** attribute:

```
def my_function():
    """This is a docstring for my_function."""
    pass

print(my_function.__doc__)

Output:- This is a docstring for my_function.
```

- Using inspect.getdoc():

```
import inspect

def my_function():
    """
        This is a docstring for my_function
        with indentation.
    """
    pass

print(inspect.getdoc(my_function))

Output:-
This is a docstring for my_function
with indentation.
```

🎯 Ques. Difference between Python comments and docstrings?

- **Comments:** Comments in Python start with a hash mark (#) and are intended to explain the code to developers. They are **ignored** by the Python interpreter.
- **Docstrings:** Docstrings provide a description of the function, method, class, or module. Unlike comments, they are **not ignored** by the interpreter and can be accessed at runtime using the .**doc** attribute.

🎯 Ques. What is python Variables?

- Variables are containers for storing data values.
- A variable name must **start** with a **letter** or the **underscore** character.
- A variable name **cannot** start with a **number**.
- Variable names are **case-sensitive** (age, Age and AGE are three different variables)

```
#Legal variable names:
myvar = "John"
MYVAR = "John"
my_var = "John"
myVar = "John"
myvar2 = "John"
_my_var = "John"

#Illegal variable names:
2myvar = "John"
my-var = "John"
my var = "John"
```

```
x = 5
y = "Mohit"
print(x)  # Output:- 5
print(y)  # Output:- Mohit
```

Single or Double Quotes:-

- single quotes (' ') and double quotes (" ") are functionally the **same and both are used to create strings**. The choice mainly depends on readability and avoiding unnecessary escaping when the string contains quotes.

```
x = "John"
print(x)  # Output:- John

# double quotes are the same as single quotes:
x = 'John'
print(x)  # Output:- John

print('He said "Hello"')  # Output:- He said "Hello"
print("It's a beautiful day")
print('It\'s a beautiful day')
print('It's a beautiful day') # Output:- error
```

Assign Multiple Values:

- Python allows you to assign values to multiple variables in one line.

```
x, y, z = "Orange", "Banana", "Cherry"

print(x)  # Output:- Orange
print(y)  # Output:- Banana
print(z)  # Output:- Cherry
```

One Value to Multiple Variables:

- we can assign the same value to multiple variables in one line.

```
x = y = z = "Orange"

print(x)  # Output:- Orange
print(y)  # Output:- Orange
print(z)  # Output:- Orange
```

- **Variables Casting:** We want to specify the data type of a variable, this can be done with casting. and We can **get the data type** of a variable with the **type()** function.

```
x = str(3)
y = int(3)
z = float(3)

print(x) # output 3
print(y)  # output 3
print(z)  # output 3.0
```

```
x = 5
y = "John"
print(type(x))
print(type(y))

Output:-
<class 'int'>
<class 'str'>
```

- **Unpack a Collection:** If we have a collection of values in a list, tuple etc. Python allows you to extract the values into variables. This is called unpacking.

```
fruits = ["apple", "banana", "cherry"]
x, y, z = fruits

print(x)
print(y)
print(z)

Output:-

apple
banana
cherry
```

Output Variables(combine both text and a variable)

```
x = "awesome"
print("Python is " + x)

output:- Python is awesome

# Example 2
-----------
x = "Python is "
y = "awesome"
z =  x + y
print(z)

output:-Python is awesome
```

```
x = 5
y = 10
print(x + y)
```

output:- 15

Note:- If you try to combine a string and a number, Python will give you an error:

```
x = 5
y = "John"
print(x + y)
output:- TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Ques. Global Variables?

- Variables that are created outside of a function.
- Global variables can be used by everyone, both inside of functions and outside.

```
x = "awesome"
def myfunc():
  print("Python is " + x)
myfunc()

output:- Python is awesome
```

```
x = "awesome"
def myfunc():
  x = "fantastic"
  print("Python is " + x)
myfunc()
print("Python is " + x)

output:-
Python is fantastic
Python is awesome
```

Ques. global Keyword?

- To create a global keyword inside a function should be treated as a global variable, you can use the **global keyword**.

```
def myfunc():
  global x
  x = "fantastic"

myfunc()
print("Python is " + x) # Output:- Python is fantastic
```

- Also, use the global keyword if you want to change a global variable inside a function.

```
x = 10  # Global variable
def modify_global():
    global x
    x = 20  # Changing the value of the global variable

modify_global()
print(x)  # This will print 20
```

Ques. How to get Id of any object?

- The id() function returns the unique identity of an object during its lifetime.
  - It returns an integer
  - This value is unique and constant for the object while it exists
  - **id()** function takes a single parameter object.

```
a = 5
print(id(a))
```

Ques. a=1, b=1 does both have same Id or not?

- Two variables in Python have same id,
- but not in lists because they are stored in different memory locations.

```
a = 10
b = 10
print(id(a))  # Output:- 9788992
print(id(b))  # Output:- 9788992
```

- In List

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a is b) # Output:- Output:- False
```

- In tuples

```
a = (1, 2, 3)
b = (1, 2, 3)
print(a is b) # Output:- True
```

What is None in Python?

- None in Python **represents the absence of a value** and is commonly used to indicate that a variable has no assigned value or a function returns nothing.
- Important Rule:- Always use is / is not with None.

```
# Always compare with None using:
if x is None:

# Avoid:
if x == None:
```

Difference between None, NaN, and Null

1. None
    1. Represents **absence of a value**
    2. An **object** of type NoneType
2. NaN (Not a Number)
    1. Represents invalid or undefined numeric value
    2. Comes from math operations
    3. Used in floating-point calculations
    4. Common in NumPy / Pandas / Data Science
3. Null
    1. NOT used in Python, Used in Java, JavaScript, PHP, C, C++

Ques. How is memory managed in Python?

- Memory management in Python is handled by the **Python Memory Manager**.
- Python also has an inbuilt garbage collector, which recycles all the unused memory and so that it can be made available to the heap space.
- Memory management in python is managed by **Python private heap space**. All Python objects and data structures are located in a private heap. The programmer does not have access to this private heap. The python interpreter takes care of this instead.

- The allocation of heap space for Python objects is done by Python’s memory manager. The core API gives access to some tools for the programmer to code.

🎯 Ques. What is Scope in Python?

- Every object in Python functions within a scope. A scope is a block of code where an object in Python remains relevant. Namespaces uniquely identify all the objects inside a program.
- Scope means **“WHERE can I use this variable?”**
- scope resolution in python follows the **LEGB rules**.
    - Local(L): Defined inside function/class
    - Enclosed(E): Defined inside enclosing functions(Nested function concept)
    - Global(G): Defined at the uppermost level
    - Built-in(B): Reserved names in Python builtin modules

1. **Global Scope/Global Variables:-** The Variable which can be **read from anywhere** in the program is known as a global scope. These variables can be accessed inside and outside the function.

```
x = 300 # global variable
def myfunc():
    print(x)

myfunc() # Outpur:- 300
print(x) # Outpur:- 300
```

2. **Local Variable**

```
def show():
    y = 5   # local variable
    print(y)

show()  # 5
print(y)   # ❌ Error - y cannot be accessed outside the function
```

3. **NonLocal or Enclosing Scope:-** Nonlocal Variable is the **variable that is defined in the nested function**. It means the variable can be neither in the local scope not in the global scope.

```
def func_outer():
    x = "local"
    def func_inner():
        nonlocal x
        x = "nonlocal"
        print("inner:", x)
    func_inner()
    print("outer:", x)
func_outer()

# Output:-inner: nonlocal
# outer: nonlocal
```

3. **Built-in Scope:-** If a Variable is not defined in local, Enclosed or global scope, then python looks for it in the built-in scope. In the Following Example, 1 from math module pi is imported, and the value of pi is not defined in global, local and enclosed. Python then looks for the pi value in the built-in scope and prints the value. Hence the name which is already present in the built-in scope should not be used as an identifier.

```
# Built-in Scope
from math import pi
# pi = 'Not defined in global pi'
def func_outer():
    # pi = 'Not defined in outer pi'
    def inner():
        # pi = 'not defined in inner pi'
        print(pi)
    inner()
func_outer()

Output:- 3.141592
```

Same Variable Name, Different Scope

```
x = 10

def test():
    x = 5   # local variable
    print(x)

test()      # 5
print(x)    # 10
```

Ques. What is the operator?

Arithmetic Operators

Operators	Descrption	Result
Addition(+)	Adds the values on either side of the operator.	3+4=7
Subtraction(-)	Subtracts the value on the right from the one on the left.	3+4=-1
Multiplication(*)	Multiplies the values on either side of the operator.	3*4=12
Division(/)	Divides the value on the left by the one on the right. Notice that division results in a floating-point value.	3/4=0.75
Exponentiation(**)	Raises the first number to the power of the second.	3**4=81
Floor Division(//)	Divides and returns the integer value of the quotient. It dumps the digits after the decimal.	10//3=3
Modulus(%)	Divides and returns the value of the remainder.	3%4=3

Comparison(Relational) Operator

Operators	Descrption	Result
Less than(<)	This operator checks if the value on the left of the operator is lesser than the one on the right.	3<4=True
Greater than(>)	It checks if the value on the left of the operator is greater than the one on the right.	3>4=False
Less than or equal to(<=)	It checks if the value on the left of the operator is lesser than or equal to the one on the right.	7<=7 = True
Greater than or equal to(>=)	It checks if the value on the left of the operator is greater than or equal to the one on the right.	0>=0 = True
Equal to(==)	This operator checks if the value on the left of the operator is equal to the one on the right.(1 is equal to the Boolean value True, but 2 isn't. Also, 0 is equal to False.)	3==3.0 = True 1==True = True 7==True = False 0==False = True 0.5==True = False

Assignment Operators

- Assignment operators

Operator	Example	Same As	Try it
=	x = 5	x = 5	x = 5 print(x) 5
+=	x += 3	x = x + 3	x = 5 x += 3 print(x) 8
-=	x -= 3	x = x - 3	x = 5 x -= 3 print(x) 2
*=	x *= 3	x = x * 3	x = 5 x *= 3 print(x) 15
/=	x /= 3	x = x/3	x = 5 x /= 3 print(x) 1.66

```
%=  x %= 3  x = x % 3
//= x //= 3 x = x // 3
**= x **= 3 x = x ** 3
&=  x &= 3  x = x & 3
|=  x |= 3  x = x | 3
^=  x ^= 3  x = x ^ 3
>>= x >>= 3 x = x >> 3
<<= x <<= 3 x = x << 3
```

Operators	Descrption	Result

Assign(=)	Assigns a value to the expression on the left. Notice that = = is used for comparing, but = is used for assigning.	>>> a=7
		>>> print(a)
		//output:- 7

Add and Assign(+=)	Adds the values on either side and assigns it to the expression on the left. a+=10 is the same as a=a+10.	>>> a+=2

Operators	Descrption	Result
>>> print(a) //output:- 9		

Divide and Assign(/=)	Divides the value on the left by the one on the right. Then it assigns it to the expression on the left.	>>> a/=7
		>>> print(a)
		//output:-1.0

Multiply and Assign(*=)	Multiplies the values on either sides. Then it assigns it to the expression on the left.	>>> a*=8
		>>> print(a)
		// 8.0

Modulus and Assign(%=)	Performs modulus on the values on either side. Then it assigns it to the expression on the left.	>>> a%=3
		>>> print(a)
		//output:-2.0

Exponent and Assign(**=)	Performs exponentiation on the values on either side. Then assigns it to the expression on the left.	>>> a**=5
		>>> print(a)
		//output:-32.0

Floor-Divide and Assign(//=)	Performs floor-division on the values on either side. Then assigns it to the expression on the left.	>>> a//=3
		>>> print(a)
		//output:-10.0

Bitwise Operators

Operators	Descrption	Result
Binary AND(&)	It performs bit by bit AND operation on the two values. Here, binary for 2 is 10, and that for 3 is 11. &-ing them results in 10, which is binary for 2.	>>> 2&3 //output:-2
Binary OR(	)	--
Binary XOR(^)	--	--
Binary One's Complement(~)	--	--
Binary Left-Shift(<<)	--	--
Binary Right-Shift(>>)	--	--

What is membership operators?

- A membership operator in Python checks if a value or variable is present in a sequence(string, list, tuples, sets, dictionary) or not.
- The output of a membership operator is a Boolean value, either True or False.
- There are two membership operators in Python.

1. In operator
2. not in' operator

in operator

- The 'in' operator is used to check if a value exists in a sequence or not.

```
#in
x = "Hello, World!"
print("ello" in x) #Returns true as it exists
print("hello" in x) #Returns false as 'h' is in lowercase
print("World" in x)#Returns true as it exists
```

```
x = ["apple", "banana"]
print("banana" in x)  # Output:- True
```

```
list1=[0,2,4,6,8]
list2=[1,3,5,7,9]

check=0
for item in list1:
    if item in list2:
        #Overlapping true so check is assigned 1
        check=1

if check==1:
    print("overlapping")
else:
    print("not overlapping")

Output:- not overlapping
```

2. **'not in operator-** Returns True if the target value is not present in a given collection of values. Otherwise, it returns False.

```
#not in
x = "Hello, World!"
print("ello" not in x) #Returns false as it exists
print("hello" not in x) #Returns true as it does not exist
print("World" not in x) #Returns false as it exists
```

```
x = ["apple", "banana"]
print("pineapple" not in x) # Output:- True
```

```
a = 70
b = 20
list = [10, 30, 50, 70, 90 ];

if ( a not in list ):
 print("a is NOT in given list")
else:
 print("a is in given list")

if ( b not in list ):
 print("b is NOT present in given list")
else:
  print("b is in given list")

Output:-
a is in given list
b is NOT present in given list
```

What is Identity operators?

- The Python Identity Operators are used to compare the objects if both the objects are actually of the **same data type** and share the **same memory location**. There are different identity operators such as: There are two Identity operators in Python.

1. is operator
2. is not operator

is operator

- is operator returns True if both variables refer to the same object, otherwise, it returns False.

```
x = 'Educative'
if (type(x) is str):
    print("true")
else:
    print("false")
Output:- True
```

is not operator

- is not operator returns True if both variables do not refer to the same object, otherwise, it returns False.

```
x = 6.3
if (type(x) is not float):
    print("true")
else:
    print("false")

Output:- False
```

```
# is not Operator:
x = ["apple", "banana"]
y = ["apple", "banana"]
z = x

print(x is not z)   # Output:- False
print(x is not y)   # Output:- True
print(x != y)   # Output:- False
```

Difference between '==' and 'is' Operator?

- The **equality operator(==)** is used to compare the value of two variables,
- whereas the identities operator **is** used to compare the memory location of two variables.

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b)   # Output:- True, because the contents are the same

# ------identities operator------
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a is b)  # False, different objects in memory
print(a is c)  # True, both refer to the same object
```

What is ternary operator in Python?

- The ternary operator in Python is a **one-line conditional expression** used to assign a value based on a condition.

```
a = 10
b = 20

max_value = a if a > b else b
print(max_value)  # Output:- 20
```

```
def check(x):
    return "Positive" if x > 0 else "Non-positive"

print(check(-5))
```

- Nested Ternary Operator

```
num = 0

result = "Positive" if num > 0 else "Negative" if num < 0 else "Zero"
print(result) # Output:= Zero
```